🔷 **What is a JOIN in SQL Server?**

A **JOIN** is used to combine rows from two or more tables based on a related column between them.

👉 Most commonly, joins are performed using:

- **Primary Key**
- **Foreign Key**

🧱 **Sample Tables (For All Examples)**

1️⃣ **Customers Table**

```
CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  CustomerName VARCHAR(50),
  City VARCHAR(50)
);
```

2️⃣ **Orders Table**

```
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustomerID INT,
  OrderAmount DECIMAL(10,2),
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

Sample Data

Customers:

CustomerID	CustomerName	City
1	Ravi	Hyderabad
2	Priya	Mumbai
3	Amit	Delhi

Orders:

OrderID	CustomerID	OrderAmount
101	1	5000
102	1	7000
103	2	3000
104	4	9000

🔶 Types of Joins in SQL Server

1️⃣ INNER JOIN

🔹 Definition

Returns only matching rows from both tables.

🔹 Syntax

```
SELECT columns

FROM Table1

INNER JOIN Table2

ON Table1.Column = Table2.Column;
```

🔹 Example

```
SELECT C.CustomerName, O.OrderAmount
```

FROM Customers C

INNER JOIN Orders O

ON C.CustomerID = O.CustomerID;

🔹 **Result**

CustomerName	OrderAmount
Ravi	5000
Ravi	7000
Priya	3000

👉 Order with CustomerID = 4 is excluded (no matching customer)

2️⃣ LEFT JOIN (LEFT OUTER JOIN)

🔹 **Definition**

Returns:

- All rows from **Left table**
- Matching rows from Right table
- If no match → NULL

🔹 **Syntax**

SELECT columns

FROM Table1

LEFT JOIN Table2

ON condition;

🔹 **Example**

SELECT C.CustomerName, O.OrderAmount

FROM Customers C

LEFT JOIN Orders O

ON C.CustomerID = O.CustomerID;

🔹 **Result**

CustomerName	OrderAmount
Ravi	5000
Ravi	7000
Priya	3000
Amit	NULL

👉 Amit has no orders → NULL

3️⃣ RIGHT JOIN (RIGHT OUTER JOIN)

🔹 **Definition**

Returns:

- All rows from **Right table**
- Matching rows from Left table
- If no match → NULL

🔹 **Example**

SELECT C.CustomerName, O.OrderAmount

FROM Customers C

RIGHT JOIN Orders O

ON C.CustomerID = O.CustomerID;

🔹 **Result**

CustomerName	OrderAmount
Ravi	5000
Ravi	7000
Priya	3000
NULL	9000

👉 CustomerID 4 exists in Orders but not in Customers → NULL

4️⃣ FULL JOIN (FULL OUTER JOIN)

🔹 **Definition**

Returns:

- All rows from both tables
- If no match → NULL on respective side

🔹 **Example**

```
SELECT C.CustomerName, O.OrderAmount

FROM Customers C

FULL JOIN Orders O

ON C.CustomerID = O.CustomerID;
```

🔹 **Result**

CustomerName	OrderAmount
Ravi	5000
Ravi	7000
Priya	3000
Amit	NULL

CustomerName	OrderAmount
NULL	9000

---

5 CROSS JOIN

◆ Definition

Returns Cartesian Product:
Every row of Table1 × Every row of Table2

◆ Syntax

SELECT *

FROM Table1

CROSS JOIN Table2;

◆ Example

If Customers = 3 rows
Orders = 4 rows

Result = 3 × 4 = 12 rows

⚠ Used rarely in business scenarios.

---

6 SELF JOIN (Very Important for Interviews)

◆ Definition

A table joined with itself.

Used when:

- Table contains hierarchical data
- Employee–Manager relationship
- Parent–Child relationship

👩‍💼 Example: Employees Table

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50),
    ManagerID INT
);
```

Sample Data

EmployeeID	EmployeeName	ManagerID
1	Raj	NULL
2	Neha	1
3	Arjun	1
4	Meena	2

🔹 Scenario: Show Employee and Their Manager Name

```
SELECT
    E.EmployeeName AS Employee,
    M.EmployeeName AS Manager
FROM Employees E
LEFT JOIN Employees M
ON E.ManagerID = M.EmployeeID;
```

🔹 Explanation

- E → Employee
- M → Manager
- Same table used twice
- Aliases are mandatory

🔹 **Result**

Employee	Manager
Raj	NULL
Neha	Raj
Arjun	Raj
Meena	Neha

🧠 How JOIN Works Internally (Conceptually)

SQL Server:

1. Reads first table
2. Reads second table
3. Applies ON condition
4. Matches rows
5. Returns result set

Execution methods:

- Nested Loop Join
- Hash Join
- Merge Join

🎯 Interview-Oriented Important Points

✓ Difference between WHERE and ON

- ON → Defines join condition
- WHERE → Filters after join

✓ LEFT JOIN with WHERE mistake

-- Wrong (converts to inner join)

WHERE O.OrderAmount IS NOT NULL

✓ Can we join more than 2 tables?

Yes.

SELECT *

FROM A

JOIN B ON condition

JOIN C ON condition

JOIN D ON condition;

📊 Quick Comparison Table

Join Type	Returns Matching	Returns Non-Matching Left	Returns Non-Matching Right
INNER	✅	❌	❌
LEFT	✅	✅	❌
RIGHT	✅	❌	✅
FULL	✅	✅	✅
CROSS	❌	❌	❌
SELF	Depends on join type		

🔥 Real-World Use Cases

Scenario	Join Type
Customer and Orders	INNER
Show all customers even without orders	LEFT
Show orphan records	RIGHT

Scenario	Join Type
Data reconciliation	FULL
Employee hierarchy	SELF
Generate combinations	CROSS

🚀 **Advanced Example (3 Tables)**

```
SELECT C.CustomerName, O.OrderID, O.OrderAmount, P.ProductName

FROM Customers C

INNER JOIN Orders O

ON C.CustomerID = O.CustomerID

INNER JOIN Products P

ON O.ProductID = P.ProductID;
```

🏁 **Final Understanding**

JOIN is used when:

- Data is normalized
- Tables are related
- Foreign keys exist
- Reporting requires combined data